Chapter 9

Textures

Originally prepared by Dr Kristian Spoerer of UNUK

Modified by Prof Sean He of UNNC

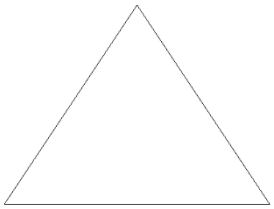
Learning Outcomes

After studying this chapter, you are expected to be able to

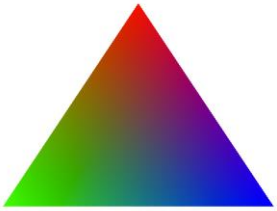
1. Define the terms texture, texel, UV, texture mapping, and texture aliasing.
2. Describe the process of texture sampling, including specifying UVs, and the Projector, Corresponder, Sample, and Transform functions.
3. Describe the problem of magnification, and how it can be avoided by using bilinear filtering.
4. Compare a magnified texture with and without bilinear filtering.
5. Explain how texture differentials are calculated and used.
6. Define what a mipmap is and explain how it is used.
7. Describe the problem of minification, and how it can be avoided using differentials, mipmaps, and trilinear filtering.
8. Compare a minified texture with and without using mipmaps and trilinear filtering.

Texture Mapping

We can render a triangle as a wireframe



We have also seen how to render a triangle with the colours of fragments mixed from the vertex colour attributes.

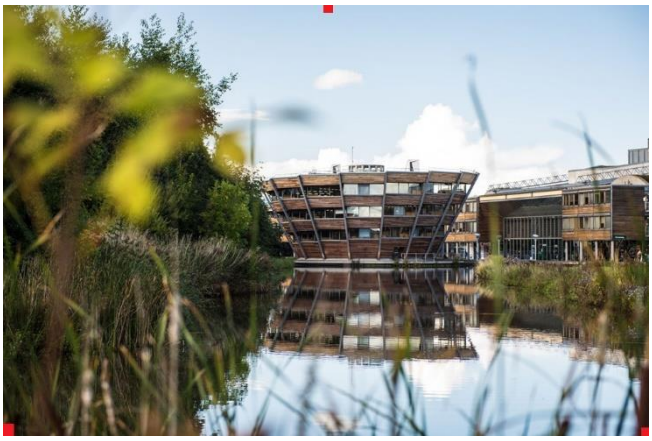


In this chapter, we will learn about adding more detail to the triangle by mapping a *texture* to the triangle.

A *texture* is an image. The following image is a texture.

A *texel* is a texture element, which is essentially a colour. For example, in this image, we can see individual texels from the texture above.

In order to map a texture onto a triangle, we need UVs. A *UV* is a location on a texture. For example, in this figure, we can see a texture with three UVs drawn at their positions with red points.

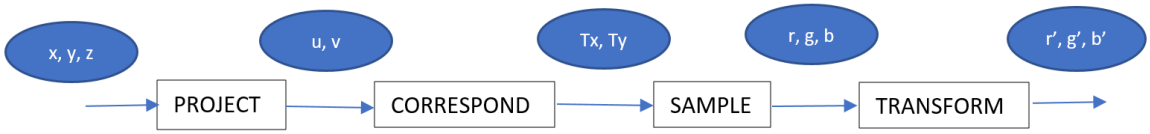


Texture mapping involves sampling a texture for each fragment on a triangle, in order to retrieve the colour from the texture, and use that colour for the fragment.

Texture Sampling

Texture Sampling involves 4 steps

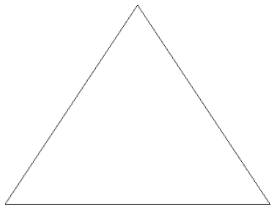
1. Project
2. Correspond
3. Sample
4. Transform



Let's say we have specified a triangle

```
float vertices[] =
{
      0.f, .5f, 0.f,
      -.5f, -.5f, 0.f,
      .5f, -.5f, 0.f
}
```

which looks like this when rendered as a wireframe



Let's say we want to draw a texture onto the triangle to give it more detail, and make the look like this

We need to specify UV values (texture coordinates) for each vertex in the array.

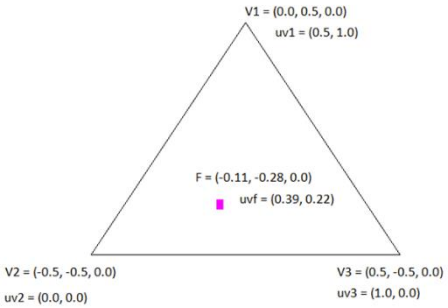
```
float vertices[] =
{
      //pos                  //UV
      0.f, .5f, 0.f,         .5f, 1.f
      -.5f, -.5f, 0.f,       0.f, 0.f,
      .5f, -.5f, 0.f,        1.f, 0.f
}
```

Then, texture sampling can follow the stages as listed above.

First, the fragment position is *projected* to a fragment UV. We already saw in chapter 2 that the barycentric coordinates can be used to interpolate the attributes from the vertices to the fragment.

In practice, this won't work for texture coordinates. Instead, perspective correct interpolation must be performed. This is more advanced and is beyond the scope of COMP3069. It is sufficient for you to know that the texture coordinate (UV) of a fragment can be interpolated from the vertices.

In the following diagram, we can see three vertices, with their positions and UVs, and a fragment drawn in magenta, with its position and UV which has been interpolated.



Next, a *corresponder* function maps the fragment UV to a texel coordinate

(0.39, 0.22) → (390.00, 146.52)

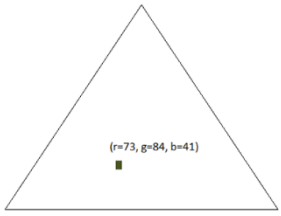
Next, a *sample* function samples the texel by dropping the fraction parts, (x=390, y=146) and reading the texel colour values (r=73, g=84, b=41)

Finally, a *transform* function maps the retrieved RGB values from the texture to ones that can be displayed

(r=73, g=84, b=41) → (r=73, g=84, b=41).

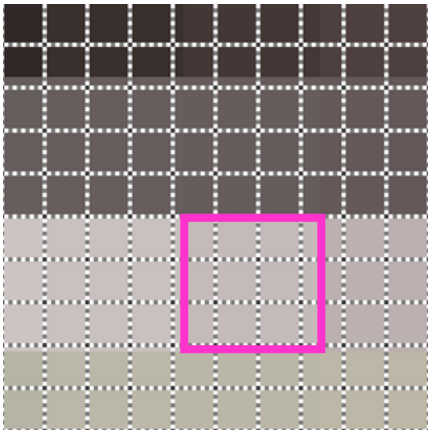
For COMP3069, we only store textures in formats that we are able to display directly, so the RGB values are unchanged.

Finally, the sampled colour can be used for the fragment.



Magnification

Magnification occurs during texture mapping when $pixels > texels$ which means that more than one pixel will sample the same texel.



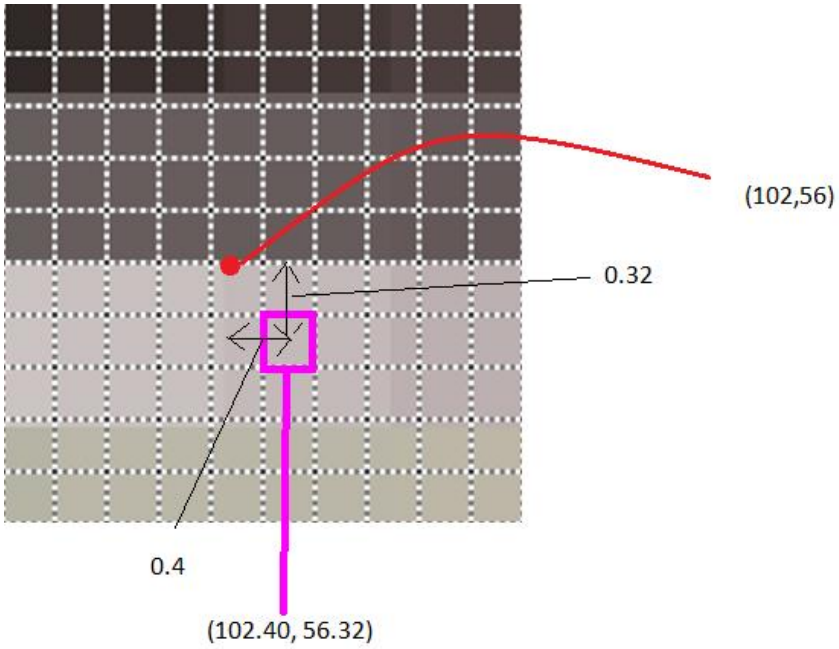
In this figure, we can see pixels as represented by the dashed grid lines, and texels one of which is outlined in magenta. In the figure, 4x3 pixels will sample the same texel.

When a texture is sampled, as described above, the corresponder function maps the fragment UV to a texel coordinate, which includes fraction parts,

UV=(0.4, 0.22) → tc=(102.40, 56.32)

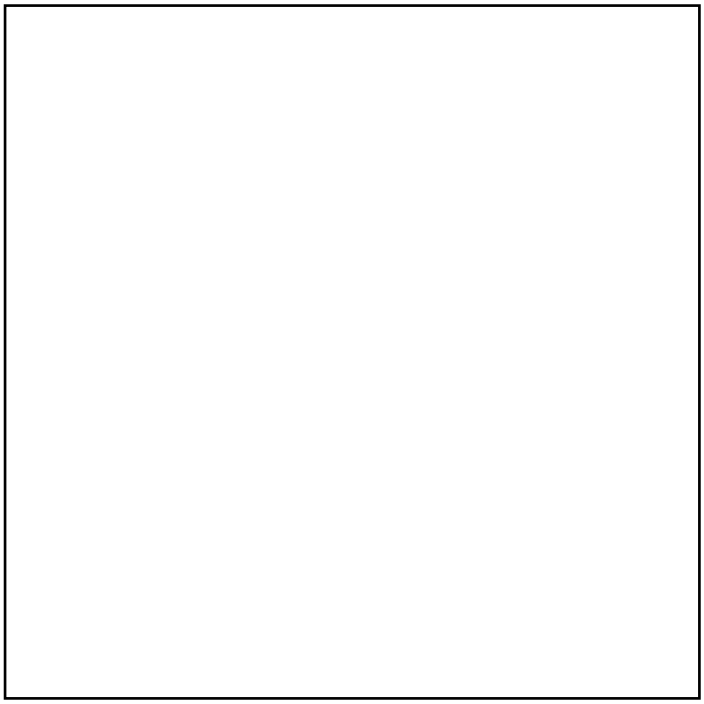
Then, the texel is sampled by dropping the fraction parts and reading the texel values

tc=(x=102, y=56) →col= (r=73, g=84, b=41).



This is kind of sampling, called *Nearest Sampling*, means that the image appears pixelated after magnification.

Let's say that we have a polygon (made of triangles) like this



We want to map the texture shown below. The diagram also shows the UV box on the sampled texture

The resulting pixelated magnification will occur

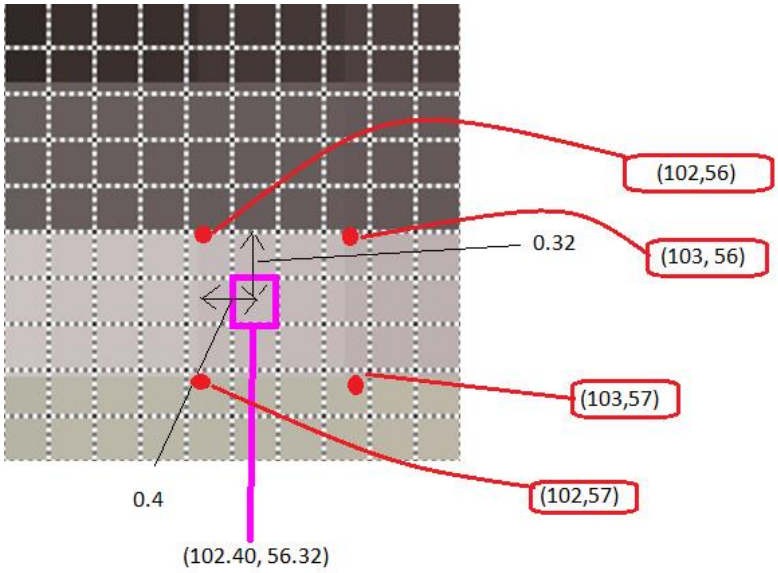
Bilinear filtering is a method for smoothing the pixilation during magnification. The nearest 4 texels are sampled, on either side of the texel coordinate including fraction parts, tc=(102.40, 56.32)

tc1=(x=102, y=56) →col= (r=73, g=84, b=41)

tc2=(x=103, y=56) →col= (r=75, g=94, b=51)

tc3=(x=102, y=57) →col= (r=105, g=94, b=31)

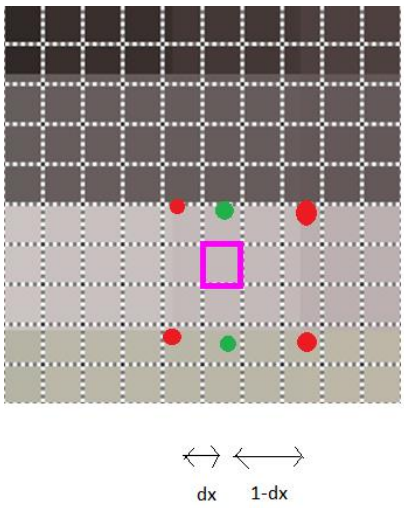
tc4=(x=103, y=57) →col= (r=107, g=97, b=37)



Then, in between, colours are interpolated in the x axis using the x fraction part dx = 0.4

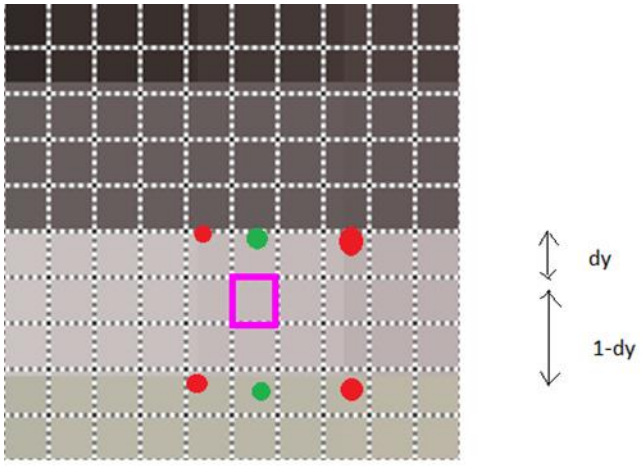
(1-0.4)*(r=73, g=84, b=41)+ 0.4*(r=75, g=94, b=51) = (73.8, 89.2, 45.0)

(1-0.4)* (r=105, g=94, b=31)+ 0.4*(r=107, g=97, b=37) = (107.8, 95.2, 33.4)

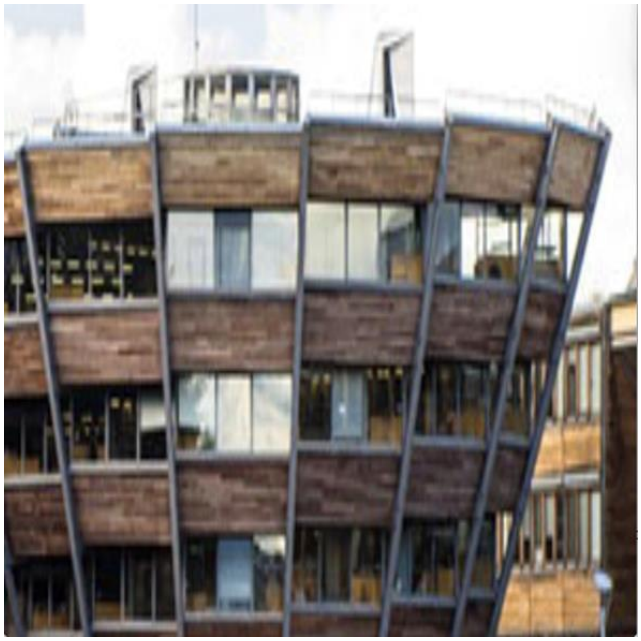


Then, the final in between colour is interpolated in the y axis, using the y fraction part dy = 0.32

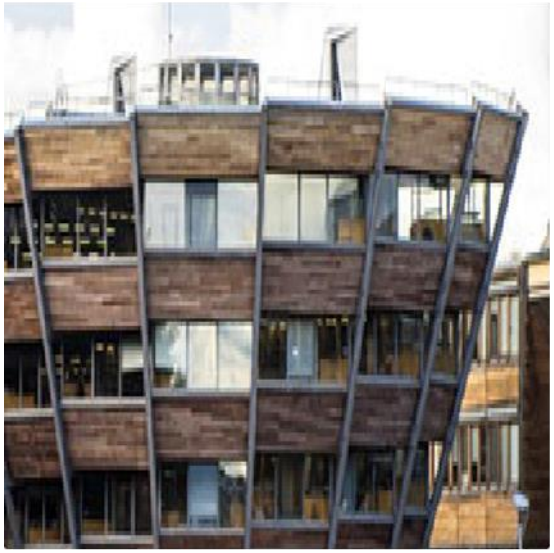
(1-0.32)* (73.8, 89.2, 45.0)+ 0.32*(107.8, 95.2, 33.4) = (r=84.68, g=91.12, b=41.29)

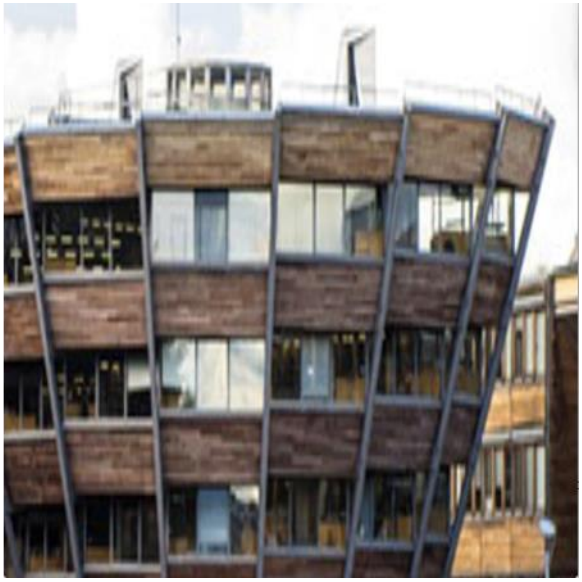


Then, if we use the same UV as above with bilinear filtering, the final result shows that the image isn’t pixelated and the magnification is smoothed.



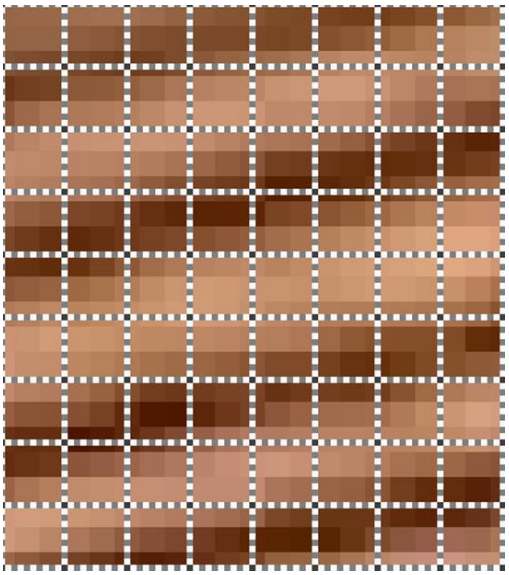
By comparison, the left image shows pixelation caused by magnification, and the right image shows smoothing of the pixelation using bilinear filtering.





Minification & Mipmaps

Minification occurs when $texels > pixels$ which means that 1 fragment will sample multiple texel colour values.

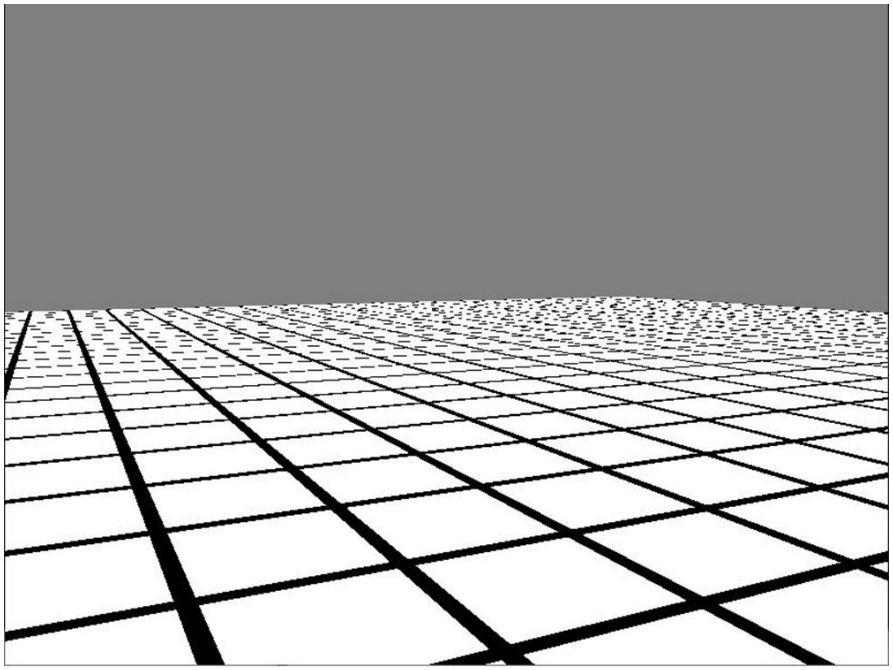


In this figure, texels can be seen of different colours, and fragments are represented using dashed grid lines. Each pixel covers more than 2x2 texels.

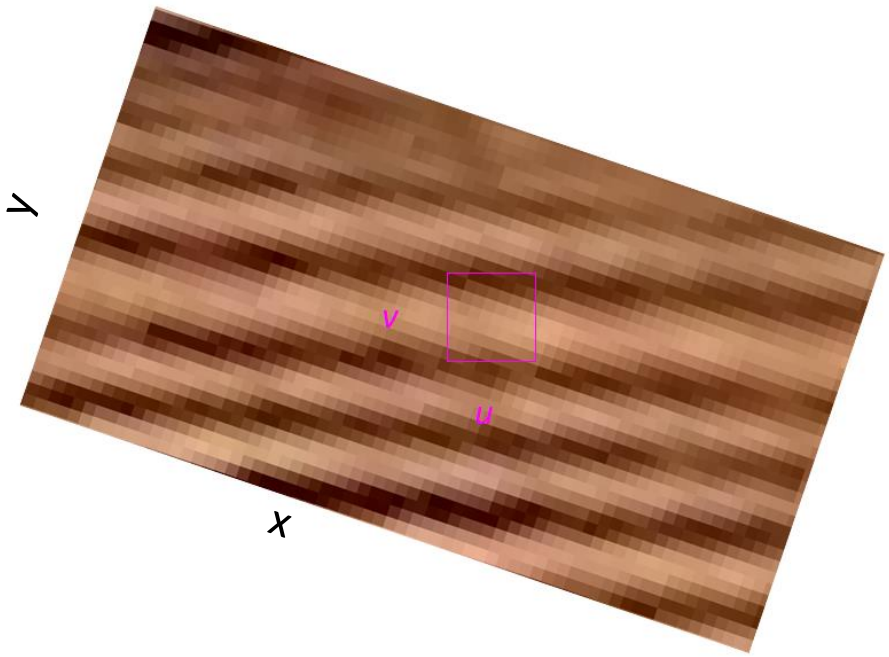
If the geometry which is textured was to move away from the camera, then the pixels/texels ratio would get even smaller.

Moreover, if the geometry which is textured was rotated on the x-axis so that the top was moved away from the camera and the bottom was moved towards the camera, then the top fragments would have a very small pixels/texels ratio, whereas the bottom fragments would have a larger pixels/texels ratio.

During texture sampling, only the nearest texel will be sampled for each fragment. If minification occurs, then a lot of colour information stored in unsampled texels will be lost. This is called *texture aliasing* and can be seen in the following image.



This can be solved by sampling all of the appropriate texels and averaging the colour for the fragment. But this leads to a new problem of how to calculate for a given fragment, how many texels should be sampled. This is solved by calculating differentials.



In this image, we can see a fragment which needs to be coloured represented by the magenta box, a texture made up of coloured texels, and also the x and y axes of the texture being sampled.

We calculate the change in x texels for one change in u texture coordinate

$\partial x/\partial u = 7.2$

the change in y texels for one change in v texture coordinate

$\partial y/\partial v = 7.2$

the change in x texels for one change in v texture coordinate

$\partial x/\partial v = 2.4$

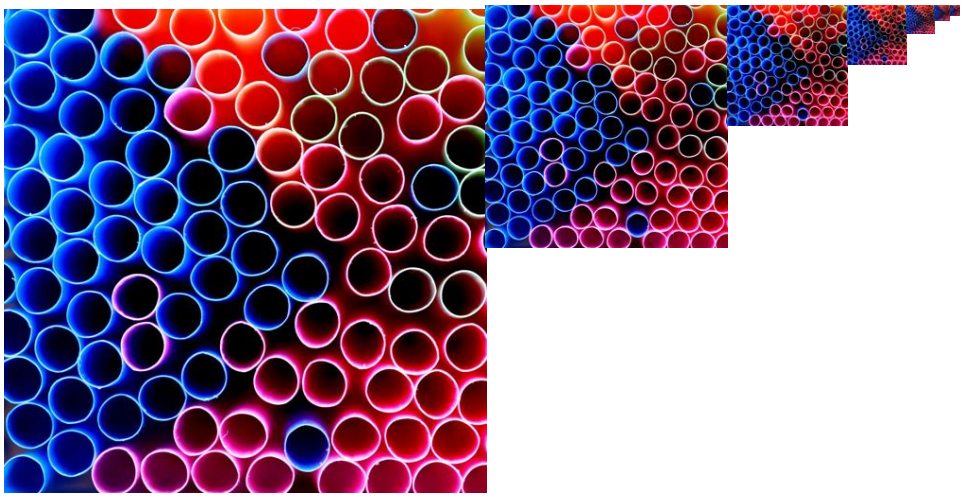
and the change in y texels for one change in u texture coordinate

$\partial y/\partial u = 2.4$

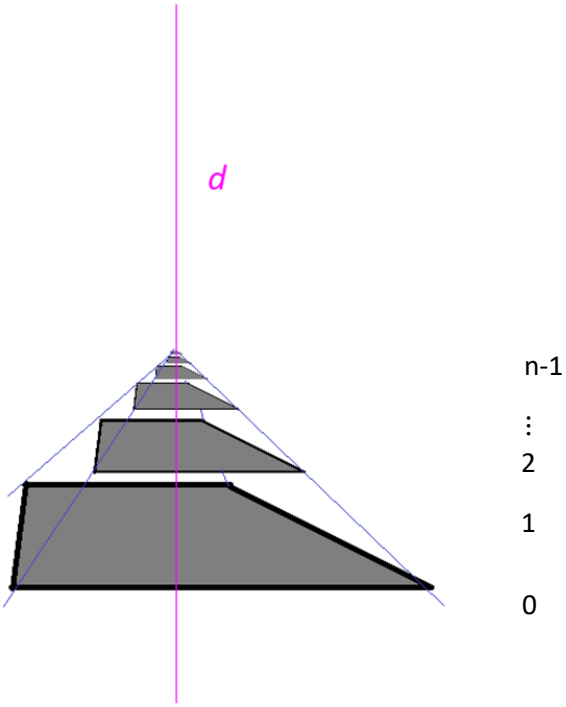
Then, take the maximum differential as the square size of the maximum number of texels covered by a fragment.

$\mathrm{d} = 7.2$

Mipmaps are a way of storing multiple copies of the same image, each half the size of the previous image in the sequence. The sequence continues until the smallest image is 1x1 pixel in size



Visualise the mipmaps as being arranged in a pyramid with the largest at the base, called level 0, and the smallest at the top, called level n-1.

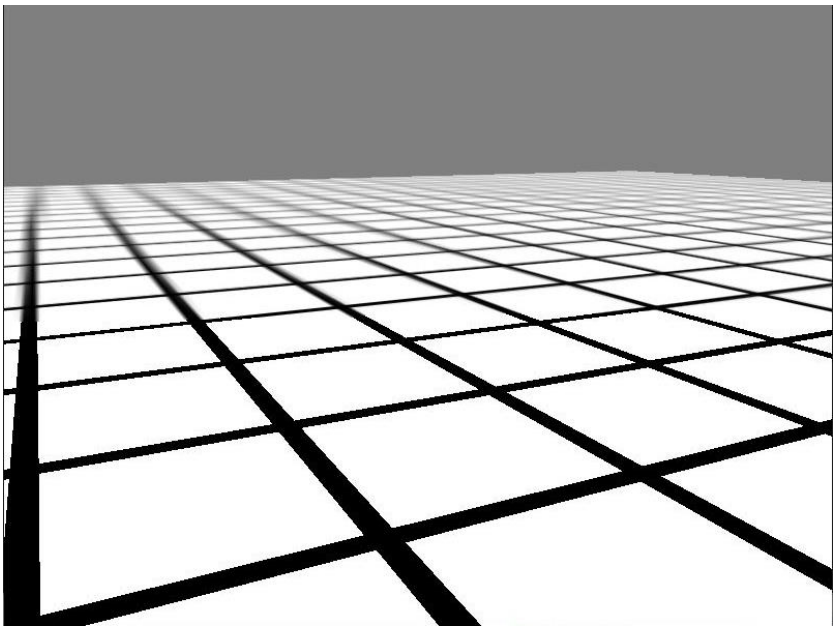


When a pixel needs to sample a colour of a texel, the d value is compared against the height in the mipmap pyramid using the following formula and the two nearest consecutive levels are used in the sampling

$$levela < \log_2 d < levelb$$

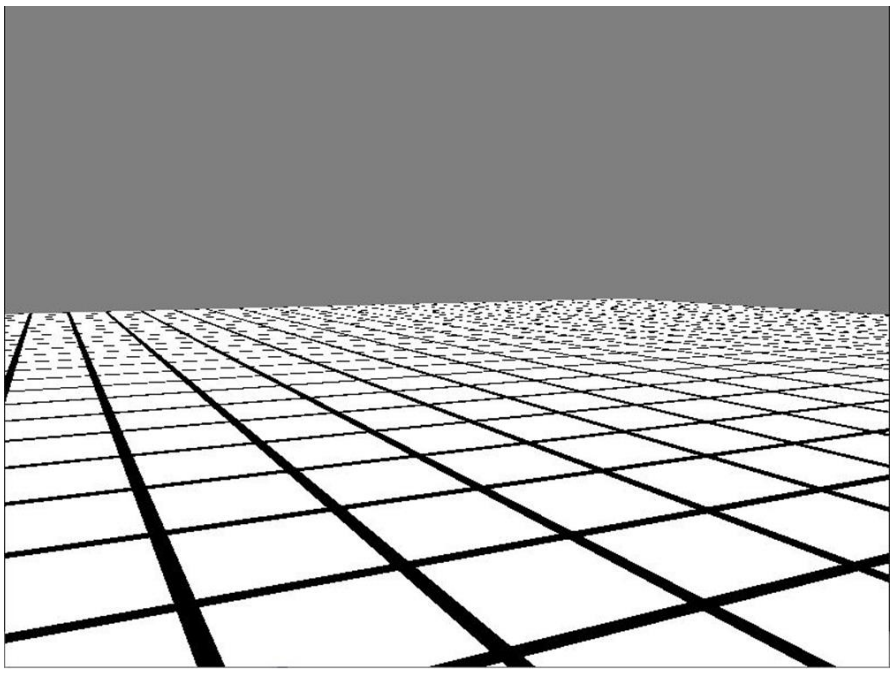
In the example, $\mathrm{d} = 7.2$, so $\log_2 7.2 = 2.85$ and the texels should be sampled from levels 2 and 3.

Bilinear sampling (as described above) samples a colour ca from from levela (level 2), and bilinear sampling samples a colour cb from levelb (level 3). Then, bilinear sampling interpolates the colours ca and cb sampled from the two levels, to make the final pixel colour. This is called *trilinear sampling*.



This image shows the smoothing of aliasing using mipmaps and trilinear sampling.

Here is the original render which includes aliasing caused by minification.



As can be seen in the comparison, using mipmaps has improved the quality of the render in the cases where minification occurs.